

AUTO-MEDS — COMPLETE E-PHARMA FULL-STACK PROJECT PROMPT

I want you to build a complete, working full-stack E-Pharma web application called "**Auto-Meds**".

The application must be built using:

- **Frontend:** Angular + TypeScript
- **Backend:** Java + Spring Boot
- **Database:** MySQL
- **Authentication:** Spring Security + JWT
- **ORM:** Spring Data JPA + Hibernate
- **Build Tool:** Maven

The application should be a real full-stack application where Angular communicates with Spring Boot REST APIs and Spring Boot persists all important data in MySQL.

Do NOT create a static frontend prototype.

Do NOT hardcode medicine, user, cart, subscription, or order data.

The main differentiating feature of the application is **chronic medication subscription and automatic refill**, while also supporting normal one-time e-pharmacy purchases.

1. PROJECT IDEA

Existing e-pharmacy applications are mainly designed for one-time medicine purchases.

For chronic patients, this creates repetitive work:

1. Search medicine
2. Add medicine to cart
3. Upload prescription
4. Checkout
5. Repeat the entire process every month

Auto-Meds should solve this problem through a subscription-first chronic medication system.

The patient can either:

ONE-TIME PURCHASE

Search medicine

- Add to Cart
- Checkout
- Place Order
- Admin processes delivery

OR:

CHRONIC MEDICATION SUBSCRIPTION

Search medicine

- Subscribe
- Upload prescription
- Admin reviews prescription
- Admin approves
- Subscription becomes ACTIVE
- System calculates refill date
- System checks stock and prescription
- Automatic refill order is generated
- Admin processes delivery
- Next refill is automatically calculated

The patient must NOT be forced to subscribe.

2. CORE FEATURES

The application must include:

Patient

- Registration
- Login
- Logout
- View all medicines
- Search medicines
- Search by medicine name
- Search by brand
- Search by composition
- Search by strength
- View medicine details

- View stock status
- Find alternative brands when medicine is unavailable
- Add medicines to cart
- Update cart quantity
- Remove medicines from cart
- Checkout
- Select delivery address
- Place one-time order
- Track one-time order
- Create medication subscription
- Upload prescription
- View subscription status
- View active subscriptions
- View upcoming refills
- Cancel subscription
- Renew subscription
- Pause/resume subscription if implemented
- View notifications
- View profile
- Logout

Admin

- Login
- Logout
- Admin dashboard
- View patients
- Add medicines
- Edit medicines
- Deactivate medicines
- Add stock
- Remove stock
- View inventory
- View low-stock medicines
- View out-of-stock medicines
- View subscription requests
- View uploaded prescriptions
- Approve subscription
- Reject subscription
- Request clarification
- Cancel subscriptions
- View active subscriptions
- View upcoming refills
- View one-time orders

- View subscription refill orders
 - Approve/reject orders
 - Update delivery status
 - Manage users
-

3. USER ROLES

There are exactly TWO roles:

□ PATIENT
ADMIN

□ Use role-based authorization.

Patients must NOT be able to access:

□ /admin/**

□ Admins must have access to admin functionality.

Admin registration must NOT be publicly available.

Create the initial admin using database initialization/data seeding.

4. TECHNOLOGY STACK

Frontend

Use:

- Angular
- TypeScript
- HTML
- CSS
- Angular Router
- Reactive Forms
- HttpClient
- Angular Services
- Route Guards

- HTTP Interceptor

Backend

Use:

- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- Spring Security
- JWT
- BCrypt
- Maven

Database

Use:

- MySQL

File Upload

Prescription uploads should initially use local server-side storage.

Design the code so cloud storage can be introduced later.

5. PATIENT HOME PAGE

After login, the patient should be redirected to:

□ [/home](#)

□ The home page should contain:

Navbar

□ [Auto-Meds](#)

[Home](#)

Medicines
Subscriptions
Cart 
Orders
Notifications
Profile
Logout

☐ The cart should show the current number of items.

Example:

☐ Cart  (3)
☐

6. PATIENT DASHBOARD

Show dashboard cards such as:

☐ Active Subscriptions
Upcoming Refill
Pending Requests
Active Orders

☐ Then display:

MY ACTIVE MEDICATIONS

Example:

☐ -----
Metformin 500mg

Brand: XYZ Pharma
Composition: Metformin
Strength: 500mg

Dosage: 1 tablet/day

Status: ACTIVE

Next Refill:
15 September 2026

Next Dispatch:
10 September 2026

[View Subscription]
[Cancel]
[Renew]

☐ If the patient has multiple active subscriptions, show all of them.

If there are no active subscriptions:

☐ No active medication subscriptions.

[Start a Subscription]

☐

7. MEDICINE CATALOG

The patient must be able to see ALL medicines stored in the database.

Create:

☐ /medicines

☐ Each medicine card should display:

- Medicine name
- Brand name
- Composition
- Strength
- Manufacturer
- Price
- Stock quantity
- In Stock / Out of Stock
- Prescription required
- View Details
- Add to Cart

- [Subscribe](#)

Example:

☐ -----

Metformin 500mg

Brand: ABC Pharma

Composition: Metformin

Strength: 500mg

Price: ₹100

Stock: 50

☒ IN STOCK

[\[View Details\]](#)

[\[Add to Cart\]](#)

[\[Subscribe\]](#)

☐ For an out-of-stock medicine:

☐ -----

Metformin 500mg

Brand: ABC Pharma

Composition: Metformin

Strength: 500mg

Price: ₹100

Stock: 0

☒ OUT OF STOCK

[\[View Details\]](#)

[\[Add to Cart - Disabled\]](#)

[\[Find Alternatives\]](#)

☐ OUT-OF-STOCK medicines must remain visible in the catalog.

8. MEDICINE SEARCH

Patients should be able to search using:

- Medicine name
- Brand name
- Composition
- Strength

Example:

Patient searches:

☐ Metformin

☐ Results could be:

☐ Metformin 500mg - ABC Pharma

Metformin 500mg - XYZ Pharma

Metformin 500mg - PQR Pharma

Metformin 1000mg - ABC Pharma

☐ Each result must show stock status.

9. SAME COMPOSITION + SAME STRENGTH ALTERNATIVE FEATURE

This is one of the most important features of Auto-Meds.

If a patient searches for or opens a medicine that is OUT OF STOCK, the system must find alternative brands.

The alternative MUST have:

☐ SAME ACTIVE COMPOSITION

+

SAME STRENGTH

☐ The brand and manufacturer may be different.

Do NOT suggest alternatives solely because the medicine name looks similar.

10. ALTERNATIVE MEDICINE EXAMPLE

Database:

☐ ID 1

Medicine: Metformin 500mg

Brand: ABC Pharma

Composition: Metformin

Strength: 500mg

Stock: 0

☐

☐ ID 2

Medicine: Metformin 500mg

Brand: XYZ Pharma

Composition: Metformin

Strength: 500mg

Stock: 50

☐

☐ ID 3

Medicine: Metformin 500mg

Brand: PQR Pharma

Composition: Metformin

Strength: 500mg

Stock: 25

☐

☐ ID 4

Medicine: Metformin 1000mg

Brand: ABC Pharma

Composition: Metformin

Strength: 1000mg

Stock: 40

☐ If ID 1 is out of stock:

Valid alternatives:

☐ ID 2

ID 3

☐ Invalid alternative:

☐ ID 4

☐ because the strength is different.

11. ALTERNATIVE MEDICINE UI

If the requested medicine is unavailable:

☐ Metformin 500mg

ABC Pharma

 OUT OF STOCK

Available Alternatives

Same composition + same strength

Metformin 500mg

XYZ Pharma

Composition: Metformin

Strength: 500mg

₹80

 IN STOCK

[\[View Details\]](#)

[\[Add to Cart\]](#)

[\[Subscribe\]](#)

Metformin 500mg

PQR Pharma

Composition: Metformin

Strength: 500mg

₹75

 IN STOCK

[\[View Details\]](#)

[\[Add to Cart\]](#)

[\[Subscribe\]](#)

☐ The patient should be able to directly add an available alternative to the cart.

12. ALTERNATIVE BACKEND API

Create:

☐ GET `/api/medicines/{medicineId}/alternatives`

☐ The backend should:

1. Find the requested medicine.
2. Read composition.
3. Read strength.
4. Search other medicines.
5. Match composition.
6. Match strength.
7. Exclude the original medicine.
8. Prefer medicines that are in stock.

9. Return alternatives.

Repository method can be:

```
❑ findByCompositionAndStrengthAndIdNot(  
    String composition,  
    String strength,  
    Long medicineId  
)
```

❑ You may also create:

❑ GET /api/medicines/alternatives?composition=Metformin&strength=500mg

❑ Example response:

```
❑ [  
  {  
    "id": 2,  
    "medicineName": "Metformin 500mg",  
    "brandName": "XYZ Pharma",  
    "composition": "Metformin",  
    "strength": "500mg",  
    "price": 80,  
    "stockQuantity": 50,  
    "inStock": true  
  }  
]  
❑
```

13. NO ALTERNATIVE AVAILABLE

If the medicine is out of stock and there is no alternative with the same composition and strength:

Show:

❑ Currently unavailable.

No alternative brand with the same composition and strength is currently available.



14. MEDICINE ENTITY

Create a Medicine entity containing at least:

`id`
`medicineName`
`brandName`
`composition`
`strength`
`category`
`price`
`stockQuantity`
`requiresPrescription`
`description`
`manufacturer`
`expiryDate`
`active`

☐ Multiple medicine records must be allowed to have:

☐ `same composition`
`same strength`
`different brands`

☐ This is necessary for the alternative medicine functionality.

15. MEDICINE DETAILS PAGE

Create:

☐ `/medicines/:id`

☐ Display:

- Medicine image
- Medicine name

- Brand
- Composition
- Strength
- Manufacturer
- Description
- Price
- Stock
- Prescription requirement

If IN STOCK:

□Quantity: [-] 1 [+]

[Add to Cart]

[Subscribe]

□If OUT OF STOCK:

□● OUT OF STOCK

[Add to Cart - Disabled]

□Then display:

□Alternative Brands

These medicines have the same composition and strength.

□Show alternatives with:

- Brand
- Composition
- Strength
- Price
- Stock

Buttons:

□[Add to Cart]

[Subscribe]

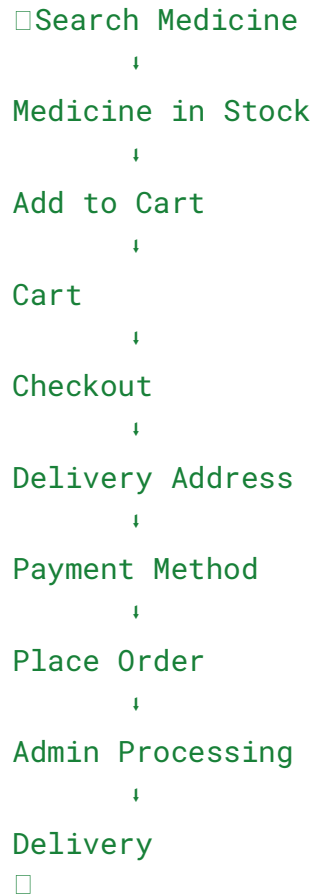
□

16. ONE-TIME PURCHASE

Not every patient wants to subscribe.

Therefore, patients must be able to make a normal one-time purchase.

Workflow:



17. ADD TO CART

For an IN-STOCK medicine:

□ [Add to Cart]

□ must be enabled.

For an OUT-OF-STOCK medicine:

□ [Add to Cart - Disabled]

□ The patient must not be able to add an unavailable medicine to the cart.

18. CART PAGE

Create:

□/cart

□The cart must show:

- Medicine
- Brand
- Price
- Quantity
- Subtotal
- Remove
- Total items
- Delivery charge
- Total amount

Example:

□MY CART

Metformin 500mg

Brand: XYZ Pharma

₹80

Quantity: [-] 2 [+]

Subtotal: ₹160

[Remove]

Amlodipine 5mg

Brand: ABC Pharma

₹50

Quantity: [-] 1 [+]

Subtotal: ₹50

[Remove]

Total Items: 3

Subtotal: ₹210

Delivery: ₹40

TOTAL: ₹250

[Proceed to Checkout]



19. CART DATABASE

Create:

Cart

Fields:

☐ id
patientId
createdAt
updatedAt

☐ CartItem

Fields:

☐ id
cartId
medicineId

quantity
priceAtAddition

□ Relationship:

□ Patient

↓

Cart

↓

CartItem

↓

Medicine

□ Each patient should have one active cart.

20. CART APIs

Create:

□ GET /api/cart

POST /api/cart/items

PUT /api/cart/items/{itemId}

DELETE /api/cart/items/{itemId}

DELETE /api/cart/clear

□ Example:

□ POST /api/cart/items

□ Request:

```
□ {  
  "medicineId": 2,  
  "quantity": 2  
}
```

□ The backend MUST verify:

1. Medicine exists.
2. Medicine is active.
3. Medicine is in stock.
4. Requested quantity is available.

Do not trust frontend stock validation.

21. CART STOCK VALIDATION

Example:

Current stock:

□ 5

□ Patient tries to add:

□ 10

□ Reject:

□ Only 5 units are currently available.

□ If cart already contains:

□ 4

□ and patient tries to add:

□ 3

□ total would become:

□ 7

□ but stock is:

□ 5

□ Reject the request.

22. CHECKOUT PAGE

Create:

☐ /checkout

☐ Show:

ORDER SUMMARY

- Medicine
- Brand
- Quantity
- Price
- Subtotal

DELIVERY ADDRESS

Patient should be able to:

- Select saved address
- Add a new address

Patient profile can contain:

☐ Name

Phone

Address

City

State

Pincode

☐ PAYMENT

For the initial project version, do NOT require a real payment gateway.

Support:

☐ Cash on Delivery

Mock/Simulated Online Payment

☐ Keep payment architecture separate so a real payment gateway can be added later.

23. CHECKOUT UI

Example:

☐CHECKOUT

ORDER SUMMARY

Metformin 500mg
XYZ Pharma
Qty: 2
₹160

Amlodipine 5mg
ABC Pharma
Qty: 1
₹50

Subtotal: ₹210
Delivery: ₹40

TOTAL: ₹250

DELIVERY ADDRESS

Aaditya Aanand
XXXXXXXXXX
Address...
City...
State...
Pincode...

[Change Address]

PAYMENT

- Cash on Delivery
- Online Payment

[PLACE ORDER]



24. CHECKOUT VALIDATION

When patient clicks:

☐ PLACE ORDER

☐ the backend must re-check:

1. User is authenticated.
2. Cart is not empty.
3. Medicines still exist.
4. Medicines are active.
5. Requested quantities are available.
6. Current prices come from backend.
7. Delivery address is valid.

IMPORTANT:

Stock must be checked AGAIN at checkout.

Do not rely on stock information from when the medicine was added to cart.

25. CART TO ORDER

Successful checkout:

```
□ Cart
  ↓
Validate Stock
  ↓
Create Order
  ↓
Create OrderItems
  ↓
Reduce Medicine Stock
  ↓
Clear Cart
  ↓
Show Order Confirmation

□ Use a database transaction.
```

26. ORDER ENTITY

Create:

```
□ Order
```

□ Fields:

```
□ id
patient
orderDate
totalAmount
deliveryAddress
paymentMethod
paymentStatus
orderStatus
orderType
expectedDeliveryDate
```

□ Order type:

```
□ ONE_TIME
SUBSCRIPTION_REFILL
```



27. ORDER ITEM ENTITY

Create:

☐ OrderItem

☐ Fields:

☐ id

order

medicine

quantity

price

subtotal



28. ONE-TIME ORDER STATUS

Use:

☐ PENDING

APPROVED

PACKED

DISPATCHED

OUT_FOR_DELIVERY

DELIVERED

CANCELLED

REJECTED



29. PATIENT ORDERS PAGE

Create:

□ /orders

□ Display:

□ Order ID

Order Type

Medicine

Quantity

Amount

Order Date

Expected Delivery

Status

□ Clicking an order:

□ /orders/:id

□ should show the complete order.

Example timeline:

□ Order Created

↓

Approved

↓

Packed

↓

Dispatched

↓

Out for Delivery

↓

Delivered

□

30. SUBSCRIPTION FEATURE

The subscription feature is the CORE feature of Auto-Meds.

It should behave similarly to an auto-payment subscription.

The patient should NOT have to manually reorder chronic medicines every month.

Patient creates a medication subscription once.

After approval:

- System calculates medication duration.
- System calculates next refill date.
- System checks prescription validity.
- System checks medicine stock.
- System creates refill order when appropriate.
- Admin processes delivery.
- Subscription continues.

31. SUBSCRIPTION ROUTES

Create a separate subscription section.

Routes:

□ `/subscriptions`

□ `/subscriptions/create`

□ `/subscriptions/:id`

□ When patient clicks:

□ `[Subscribe]`

□ navigate to:

□ `/subscriptions/create`

□ Do NOT implement the entire subscription flow only as a popup.

32. SUBSCRIPTION FORM

The subscription page should contain:

□ `Medicine`

`Brand`

`Composition`

`Strength`

Dosage
Frequency
Quantity
Prescription Upload
Review
Submit Request

☐ Example:

☐ CREATE MEDICATION SUBSCRIPTION

Medicine:
Metformin 500mg

Brand:
XYZ Pharma

Composition:
Metformin

Strength:
500mg

Dosage:
1 tablet

Frequency:
Once Daily

Quantity:
30 tablets

Prescription:
[Choose File]

[Submit Subscription Request]



33. PRESCRIPTION UPLOAD

Prescription upload must be part of the subscription request.

Patient should NOT create a subscription first and upload the prescription separately.

The request should contain:

□ Subscription Details

+

Prescription

□ Allowed formats:

□ PDF

JPG

JPEG

PNG

□ Validate:

- File type
 - File size
-

34. PRESCRIPTION ENTITY

Create:

□ Prescription

□ Fields:

□ id

patientId

subscriptionRequestId

fileName

filePath

uploadDate

expiryDate

status

□ Status:

□ PENDING

APPROVED

REJECTED

CLARIFICATION_REQUIRED

EXPIRED

□

35. ADMIN SUBSCRIPTION REQUESTS

Create:

□ /admin/subscription-requests

□ Admin should see:

□

Patient	Medicine	Date	Status
Aaditya	Metformin	08 Aug	PENDING
Rahul	Amlodipine	08 Aug	PENDING

□ Click:

□ /admin/subscription-requests/:id

□ Display:

- Patient
- Medicine
- Brand
- Composition
- Strength
- Dosage
- Frequency
- Quantity
- Prescription
- Request date
- Status

Buttons:

☐ [APPROVE]
[REJECT]
[REQUEST CLARIFICATION]
☐

36. PRESCRIPTION REVIEW

Admin must be able to securely view/download the prescription.

Admin can:

☐ APPROVE
REJECT
REQUEST CLARIFICATION

☐ If clarification is required, admin can enter:

☐ Please upload a newer prescription.

☐ Patient should see this message.

37. SUBSCRIPTION ENTITY

Create:

☐ Subscription

☐ Fields should include:

☐ id
patient
medicine
prescription
dosage
frequency
quantity
startDate
nextRefillDate
nextDispatchDate

status
createdAt
updatedAt

☐ Statuses:

☐ PENDING
ACTIVE
REJECTED
CANCELLED
EXPIRED
PRESCRIPTION_EXPIRED
CLARIFICATION_REQUIRED
PAUSED
☐

38. ACTIVE SUBSCRIPTIONS

If the patient has active subscriptions, display them on the home page.

Example:

☐ MY ACTIVE MEDICATIONS

Metformin 500mg

Brand: XYZ Pharma

Dosage:
1 tablet/day

Status:
● ACTIVE

Next Refill:
15 September

Next Dispatch:
10 September

[View Subscription]
[Cancel]
[Renew]

□

39. AUTO REFILL LOGIC

This is the MOST IMPORTANT backend functionality.

Example:

□Medicine:
Metformin 500mg

Quantity:
30 tablets

Dosage:
1 tablet/day

□Medication duration:

□ $30 / 1 = 30$ days

□If subscription begins:

□15 August 2026

□Next refill:

□14 September 2026

□Use:

□REFILL_BUFFER_DAYS = 5

□Therefore dispatch preparation:

□ 9 September 2026

□ The system should prepare the refill 5 days before the medicine runs out.

40. SPRING BOOT SCHEDULER

Use:

□ @Scheduled

□ Create a scheduled refill process.

Every day it should check:

- Active subscriptions
- Next refill dates
- Prescription validity
- Medicine stock

For each ACTIVE subscription:

1. Determine whether refill is approaching.
 2. Check prescription validity.
 3. Check stock.
 4. Find alternatives if medicine is unavailable.
 5. Create refill order/request if allowed.
 6. Notify patient/admin.
-

41. PRESCRIPTION EXPIRY

Track prescription validity.

Example:

□ Prescription expires:

30 August

□ Notify:

□ 30 days before

15 days before

7 days before

☐After expiry:

☐Subscription status:

PRESCRIPTION_EXPIRED

☐Automatic refill must STOP.

Patient must upload a new prescription.

42. STOCK-OUT DURING AUTOMATIC REFILL

Before creating an automatic refill:

Check medicine stock.

If available:

☐Create refill order.

☐If unavailable:

DO NOT silently fail.

Search:

☐SAME COMPOSITION

+

SAME STRENGTH

☐If alternatives exist:

Notify patient:

☐Your subscribed medicine Metformin 500mg - ABC Pharma is currently out of stock.

Available alternatives with the same composition and strength:

XYZ Pharma

PQR Pharma

☐ Patient can select an alternative.

Do NOT silently substitute a different medicine.

43. SUBSCRIPTION ALTERNATIVE FLOW

Example:

Patient has:

☐ Subscription:

Metformin 500mg

Brand: ABC Pharma

☐ Before refill:

☐ ABC Pharma stock = 0

☐ System:

☐ Detect stock-out

↓

Search same composition

↓

Match same strength

↓

Find available brands

↓

Notify patient

↓

Patient selects alternative

↓

Admin confirms if required

↓

Upcoming refill uses selected medicine

☐ Maintain a record of the original medicine and selected alternative.

44. CANCEL SUBSCRIPTION

Patient can cancel an active subscription.

Show:

☐ Are you sure you want to cancel this medication subscription?

☐ After confirmation:

☐ Subscription = CANCELLED

☐ No future automatic refill should be generated.

Existing orders should follow their existing order/cancellation rules.

45. RENEW SUBSCRIPTION

Patient can renew expired/cancelled subscriptions.

Click:

☐ [Renew Subscription]

☐ If prescription has expired:

Ask patient to upload a new prescription.

If valid:

Continue according to business rules.

46. PAUSE/RESUME SUBSCRIPTION

Optionally implement:

☐ PAUSE
RESUME

☐ When paused:

No automatic refill should be generated.

Patient can resume later.

47. INVENTORY / WAREHOUSE

Create:

☐ `/admin/inventory`

☐ Admin can:

- Add medicine
- Edit medicine
- Deactivate medicine
- Increase stock
- Decrease stock
- View current stock
- Search medicine
- Filter inventory

Filters:

☐ In Stock

Low Stock

Out of Stock

☐ Prefer:

☐ `active = false`

☐ instead of permanently deleting medicine records.

48. ADMIN MEDICINE MANAGEMENT

Create:

☐ `/admin/medicines`

☐ Admin can:

- Add medicine
 - Edit medicine
 - Deactivate medicine
 - View medicine
 - Update price
 - Update composition
 - Update strength
 - Update manufacturer
 - Update expiry date
 - Update prescription requirement
-

49. ADMIN DASHBOARD

Create:

□ /admin/dashboard

□ Display:

□ Total Patients

Total Medicines

Active Subscriptions

Pending Subscription Requests

Upcoming Refills

Pending Orders

Low Stock Medicines

Out of Stock Medicines

□ Example:

□ -----

Patients	250
----------	-----

Medicines	500
-----------	-----

Active Subscriptions	125
----------------------	-----

Pending Requests	12
------------------	----

Upcoming Refills	20
------------------	----

Pending Orders	8
----------------	---

Low Stock	15
-----------	----

Out of Stock

5

Below:

Pending Subscription Requests

Upcoming Refills

Low Stock Medicines

Pending Deliveries

50. ADMIN ORDER MANAGEMENT

Create:

/admin/orders

Admin must see BOTH:

ONE-TIME ORDERS

and:

SUBSCRIPTION REFILL ORDERS

Example:

ORDER TYPE	PATIENT	STATUS
ONE_TIME	Aaditya	PENDING
SUBSCRIPTION_REFILL	Rahul	APPROVED
ONE_TIME	Priya	DISPATCHED

Admin can:

- Approve
- Reject
- Pack
- Dispatch
- Mark out for delivery

- Mark delivered
 - Cancel
-

51. ORDER TYPES

The Order entity must distinguish:

☐ ONE_TIME

SUBSCRIPTION_REFILL

☐ Example:

☐ Order 101:

ONE_TIME

☐

☐ Order 102:

SUBSCRIPTION_REFILL

☐ ONE_TIME orders come from the cart.

SUBSCRIPTION_REFILL orders are generated by the subscription/refill system.

52. NOTIFICATION SYSTEM

Create an in-app notification system.

Patient notifications:

- Subscription submitted
- Subscription approved
- Subscription rejected
- Clarification requested
- Prescription expiring
- Prescription expired
- Refill approaching
- Medicine out of stock
- Alternative medicine available
- Order approved
- Order dispatched

- Order delivered
- Subscription cancelled

Admin notifications:

- New subscription request
 - New prescription
 - Upcoming refill
 - Low stock
 - Out-of-stock medicine
 - Patient selected alternative
 - New one-time order
-

53. AUTHENTICATION

Implement:

□Registration

Login

Logout

□Use:

□Spring Security

JWT

BCrypt

□Patient registration:

□Name

Email

Password

Phone

Address

City

State

Pincode

□Admin must not be publicly registered.

Create initial admin using data initialization.

Passwords must be hashed using BCrypt.

54. JWT

After login:

□ Angular

↓

POST /api/auth/login

↓

Spring Boot

↓

Validate credentials

↓

Generate JWT

↓

Angular stores token

□ Use Angular HTTP interceptor to attach:

□ Authorization: Bearer <token>

□ to protected API requests.

Create:

□ AuthGuard

RoleGuard

□

55. ANGULAR ROUTES

Create:

□ /login

/register

/home

```
/medicines  
/medicines/:id
```

```
/cart  
/checkout
```

```
/subscriptions  
/subscriptions/create  
/subscriptions/:id
```

```
/orders  
/orders/:id
```

```
/profile
```

```
/admin  
/admin/dashboard  
/admin/medicines  
/admin/inventory  
/admin/subscription-requests  
/admin/subscription-requests/:id  
/admin/orders  
/admin/users
```

☐ Protect admin routes.

56. BACKEND PACKAGE STRUCTURE

Use:

☐ `com.automeds`

```
|— config  
|— controller  
|— service  
|— repository  
|— entity
```

- dto
- security
- exception
- scheduler
- util

☐ Use DTOs.

Do NOT directly expose JPA entities from controllers.

57. ENTITIES

Create at minimum:

- ☐ User
- Medicine
- Prescription
- Subscription
- Order
- OrderItem
- Cart
- CartItem
- Notification

☐ Add other entities if necessary.

58. DATABASE RELATIONSHIPS

User:

- ☐ User
 - Cart
 - Subscriptions
 - Orders
 - Prescriptions
 - Notifications

☐ Medicine:

- Medicine
 - └─ Subscriptions
 - └─ OrderItems

- Subscription:

- Subscription
 - └─ User
 - └─ Medicine
 - └─ Prescription
 - └─ Orders

- Prescription:

- Prescription
 - └─ Subscription

- Cart:

- Cart
 - └─ CartItems

- CartItem:

- CartItem
 - └─ Medicine

- Order:

- Order
 - └─ User
 - └─ Subscription
 - └─ OrderItems

- Make sure JPA relationships do not cause circular JSON serialization.

59. REST API DESIGN

Authentication

- POST /api/auth/register

- POST /api/auth/login

❑ Medicines

❑ GET /api/medicines
GET /api/medicines/{id}
GET /api/medicines/search?query=
GET /api/medicines/{id}/alternatives

❑ Cart

❑ GET /api/cart
POST /api/cart/items
PUT /api/cart/items/{itemId}
DELETE /api/cart/items/{itemId}
DELETE /api/cart/clear

❑ Checkout

❑ POST /api/orders/checkout

❑ Subscriptions

❑ POST /api/subscriptions
GET /api/subscriptions/my
GET /api/subscriptions/{id}
PUT /api/subscriptions/{id}/cancel
PUT /api/subscriptions/{id}/renew
PUT /api/subscriptions/{id}/pause
PUT /api/subscriptions/{id}/resume

❑ Prescriptions

❑ POST /api/subscriptions/{id}/prescription
GET /api/prescriptions/{id}

❑ Patient Orders

❑ GET /api/orders/my
GET /api/orders/{id}

□ Admin Dashboard

□ GET /api/admin/dashboard

□ Admin Subscriptions

□ GET /api/admin/subscriptions/requests

GET /api/admin/subscriptions/requests/{id}

PUT /api/admin/subscriptions/{id}/approve

PUT /api/admin/subscriptions/{id}/reject

PUT /api/admin/subscriptions/{id}/clarification

□ Admin Medicines

□ POST /api/admin/medicines

PUT /api/admin/medicines/{id}

DELETE /api/admin/medicines/{id}

□ Admin Inventory

□ GET /api/admin/inventory

PUT /api/admin/inventory/{medicineId}/stock

□ Admin Orders

□ GET /api/admin/orders

PUT /api/admin/orders/{id}/approve

PUT /api/admin/orders/{id}/reject

PUT /api/admin/orders/{id}/status

□ Admin Users

□ GET /api/admin/users

□

60. BUSINESS RULES

Implement these rules exactly.

Medicine

1. Patient can see ALL medicines.
2. Patient can search medicines.
3. Patient can search by name, brand, composition and strength.
4. Patient can see stock status.
5. Out-of-stock medicines remain visible.
6. Out-of-stock medicines cannot be added to cart.
7. Out-of-stock medicines should show alternatives.

Alternative Medicine

8. Alternatives must have the SAME composition.
9. Alternatives must have the SAME strength.
10. Different brand/manufacturer is allowed.
11. Different strength is NOT allowed.
12. Different composition is NOT allowed.
13. Prefer alternatives that are in stock.
14. Patient can add available alternatives to cart.
15. Patient can subscribe to available alternatives.
16. Never silently substitute a medicine.

Cart

17. Cart is ONLY for one-time purchases.
18. Subscription medicines should NOT be stored in the normal cart.
19. Backend must validate stock when adding to cart.
20. Backend must validate stock again at checkout.
21. Checkout must be transactional.
22. Stock should be reduced after successful order creation.
23. Cart should be cleared after successful checkout.

Subscription

24. Subscription is a separate workflow.
25. Subscription has a dedicated route.
26. Prescription upload is part of subscription request.
27. Admin must approve the subscription.
28. Subscription becomes ACTIVE only after approval.
29. Active subscriptions appear on patient home page.
30. System calculates refill dates automatically.
31. Default refill buffer is 5 days.
32. Prescription validity must be checked.

33. Expired prescriptions stop automatic refills.
34. Patient can cancel subscription.
35. Patient can renew subscription.
36. Patient may pause/resume subscription.

Auto Refill

37. Scheduler checks active subscriptions.
38. Scheduler checks refill dates.
39. Scheduler checks prescription validity.
40. Scheduler checks medicine stock.
41. If medicine is unavailable, find same composition + strength alternatives.
42. Notify patient.
43. Do not silently replace medicine.
44. Generate subscription refill orders when appropriate.

Admin

45. Admin manages medicines.
 46. Admin manages warehouse stock.
 47. Admin manages subscription requests.
 48. Admin reviews prescriptions.
 49. Admin manages one-time orders.
 50. Admin manages subscription refill orders.
 51. Admin manages delivery status.
-

61. SECURITY

Implement:

- JWT authentication
- BCrypt password hashing
- Role-based authorization
- @Valid validation
- Global exception handling
- CORS
- Secure prescription access
- File type validation
- File size validation
- No password exposure
- No sensitive information in API responses

Prescription files must NOT be exposed through an unrestricted public folder.

Create secure endpoints for viewing/downloading prescriptions.

62. UI DESIGN

Create a clean, modern, professional pharmacy interface.

Use:

- White/light background
- Blue/green pharmacy-style accents
- Cards
- Tables
- Status badges
- Responsive navbar
- Admin sidebar
- Dashboard cards
- Search bar
- Forms
- Confirmation dialogs
- Responsive layout

Status indicators:

- ☐  IN STOCK
-  OUT OF STOCK
-  ACTIVE
-  PENDING
-  APPROVED
-  REJECTED
-  CANCELLED
-  EXPIRED

☐ For alternatives clearly display:

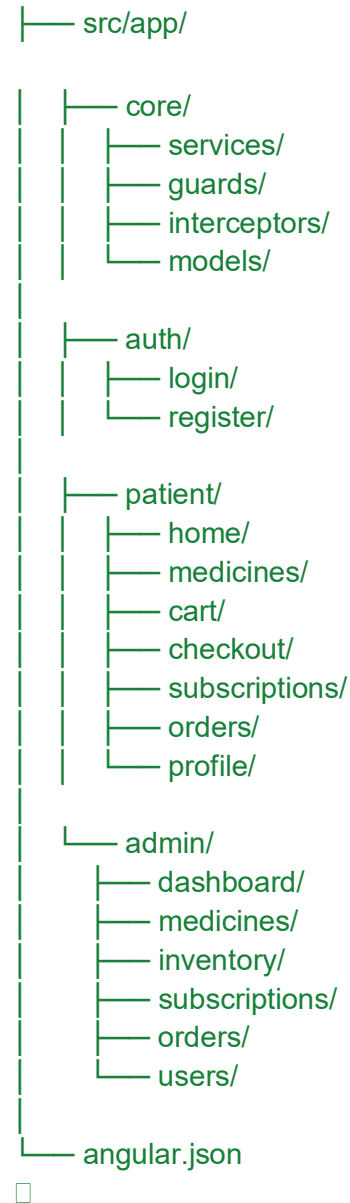
☐ Same composition + same strength

☐ so patients understand why the medicine is being suggested.

63. FRONTEND PROJECT STRUCTURE

Use:

❏ auto-meds-frontend/



64. BACKEND PROJECT STRUCTURE

Use:

❏ auto-meds-backend/

|— src/main/java/com/automed/

| |— config/
| |— controller/
| |— service/
| |— repository/
| |— entity/
| |— dto/
| |— security/
| |— exception/
| |— scheduler/
| |— util/

|— src/main/resources/
| |— application.properties
| |— data.sql

|— pom.xml

❏

65. MYSQL CONFIGURATION

Use MySQL.

Provide:

- Database creation command
- application.properties
- JPA configuration
- Database schema
- Sample data
- Initial admin
- Initial patient

Sample medicine data MUST demonstrate alternatives.

For example:

❏ Metformin 500mg

ABC Pharma

```
Metformin
500mg
Stock = 0
☐
☐Metformin 500mg
XYZ Pharma
Metformin
500mg
Stock = 50
☐
☐Metformin 500mg
PQR Pharma
Metformin
500mg
Stock = 25
☐
☐Metformin 1000mg
ABC Pharma
Metformin
1000mg
Stock = 40

☐Testing:

☐Search Metformin 500mg
      ↓
ABC Pharma OUT OF STOCK
      ↓
Show XYZ Pharma
Show PQR Pharma
      ↓
Do NOT show Metformin 1000mg
☐
```

66. INITIAL USERS

Create an initial ADMIN account through data initialization.

Create a sample PATIENT account.

Admin must not be publicly registered.

Clearly provide testing credentials.

Use BCrypt-hashed passwords.

67. IMPORTANT WORKFLOW — ONE-TIME PURCHASE

The complete one-time purchase workflow must be:

□PATIENT LOGIN

↓

HOME

↓

SEARCH MEDICINE

↓

MEDICINE DETAILS

↓

CHECK STOCK

↓

MEDICINE IN STOCK

↓

ADD TO CART

↓

CART

↓

UPDATE QUANTITY

↓

CHECKOUT

↓

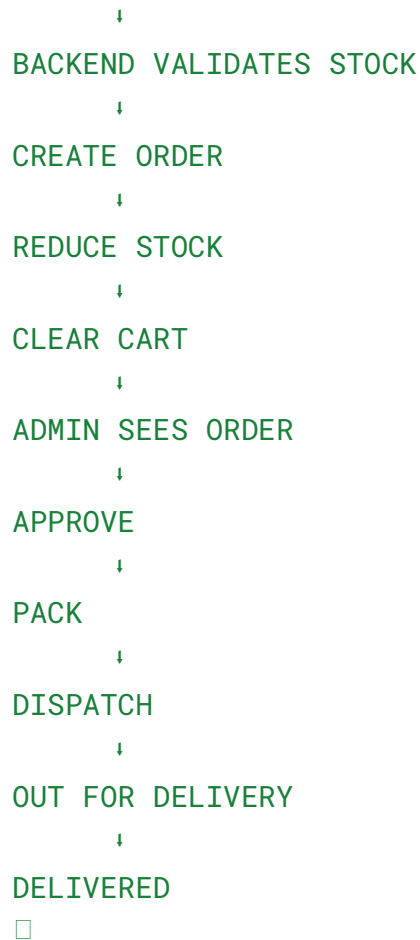
DELIVERY ADDRESS

↓

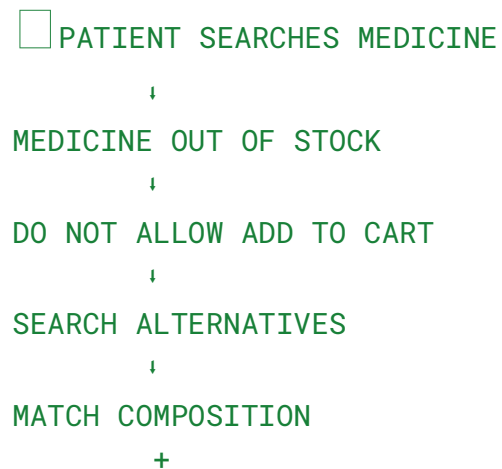
PAYMENT METHOD

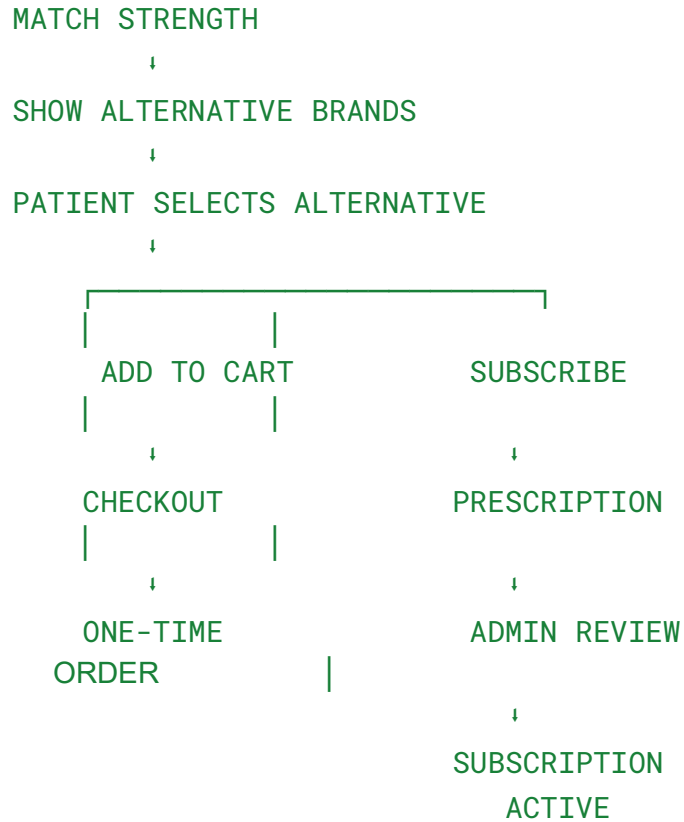
↓

PLACE ORDER



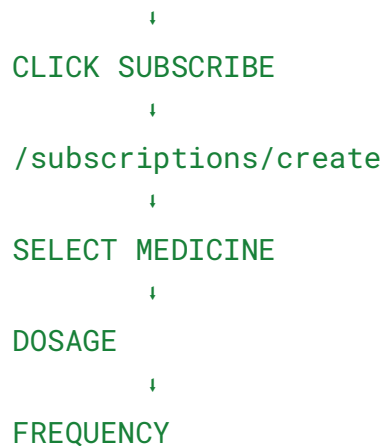
68. IMPORTANT WORKFLOW — OUT-OF-STOCK

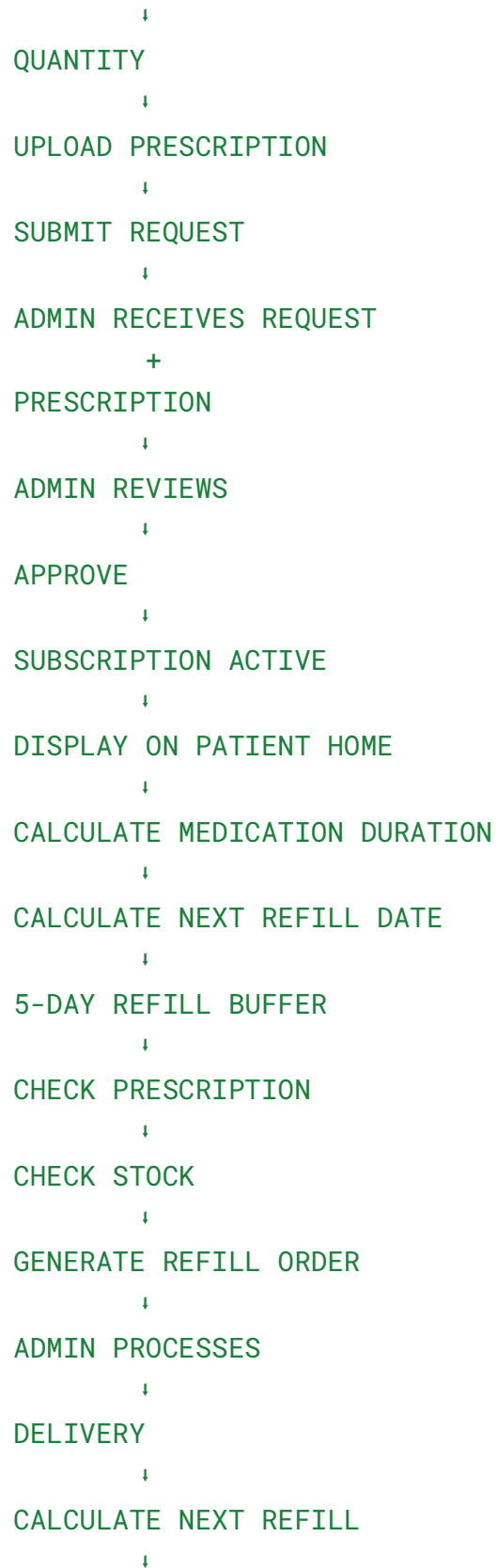




69. IMPORTANT WORKFLOW — SUBSCRIPTION

 PATIENT SEARCHES MEDICINE





REPEAT



70. IMPORTANT WORKFLOW — SUBSCRIPTION STOCK-OUT

 ACTIVE SUBSCRIPTION

↓

REFILL DATE APPROACHING

↓

CHECK STOCK

↓

MEDICINE OUT OF STOCK

↓

FIND SAME COMPOSITION

+

SAME STRENGTH

↓

ALTERNATIVES FOUND

↓

NOTIFY PATIENT

↓

PATIENT SELECTS ALTERNATIVE

↓

ADMIN CONFIRMS IF REQUIRED

↓

CREATE REFILL ORDER

↓

DELIVERY

 If no alternative exists:

 Notify patient/admin

No automatic substitution

Do not generate an invalid refill



71. COMPLETE PATIENT NAVIGATION

Patient navbar:

☐ Auto-Meds

Home

Medicines

Subscriptions

Cart 

Orders

Notifications

Profile

Logout



72. COMPLETE ADMIN NAVIGATION

Admin sidebar:

☐ Auto-Meds Admin

Dashboard

Medicines

Inventory

Subscription Requests

Active Subscriptions

Orders

Patients

Notifications

Logout



73. DEVELOPMENT ORDER

Do NOT dump the entire project into one giant response.

Build the project step-by-step.

FIRST provide:

1. Complete system architecture
2. Architecture diagram
3. Database ER diagram
4. Database schema
5. Entity relationships
6. Backend package structure
7. Angular project structure
8. API list
9. Authentication flow
10. Medicine search flow
11. Alternative medicine flow
12. Cart flow
13. Checkout flow
14. Subscription flow
15. Prescription flow
16. Auto-refill flow
17. Order/delivery flow

THEN implement the backend.

For every backend file provide:

- Exact file path
- Complete code
- Imports
- Explanation
- Dependencies

Then implement the Angular frontend.

For every frontend file provide:

- Exact file path
- Complete TypeScript
- Complete HTML
- Complete CSS
- Routing
- Services

- Guards
- Interceptors

Do NOT provide incomplete snippets.

74. REAL DATABASE REQUIREMENT

The application must NOT use hardcoded data.

These must be persisted in MySQL:

□Users

Medicines

Inventory

Cart

Cart Items

Subscriptions

Prescriptions

Orders

Order Items

Notifications

□Angular must communicate with Spring Boot REST APIs.

Spring Boot must communicate with MySQL.

75. TESTING CHECKLIST

At the end provide a complete testing checklist.

Authentication

- Patient registration
- Patient login
- Patient logout
- Admin login
- Invalid login
- JWT validation

- Role protection

Medicines

- View all medicines
- Search by name
- Search by brand
- Search by composition
- Search by strength
- View stock
- Out-of-stock medicine
- Same composition alternatives
- Same strength validation
- Different strength should NOT appear as alternative

Cart

- Add medicine
- Increase quantity
- Decrease quantity
- Remove medicine
- Empty cart
- Stock validation
- Checkout validation

Checkout

- Delivery address
- Payment method
- Place order
- Order creation
- Stock reduction
- Cart clearing

Subscription

- Create subscription
- Upload prescription
- Admin receives request
- Admin approves
- Admin rejects
- Admin requests clarification

- Patient sees status
- Active subscription appears on home
- Cancel subscription
- Renew subscription
- Pause subscription
- Resume subscription

Auto Refill

- Calculate medication duration
- Calculate next refill
- Five-day buffer
- Prescription validation
- Prescription expiry
- Stock validation
- Automatic refill order
- Out-of-stock detection
- Alternative medicine detection
- Patient confirmation
- Admin confirmation

Inventory

- Add medicine
- Edit medicine
- Deactivate medicine
- Add stock
- Remove stock
- Low stock
- Out of stock

Orders

- One-time order
- Subscription refill order
- Admin approval
- Reject
- Pack
- Dispatch
- Out for delivery
- Delivered
- Cancelled

76. FINAL PROJECT OBJECTIVE

The final application must clearly demonstrate that Auto-Meds is a complete E-Pharma platform with TWO major capabilities.

CAPABILITY 1 — NORMAL E-PHARMA SHOPPING

- Search

- Check Stock
- Add to Cart
- Checkout
- One-Time Order
- Delivery

□ CAPABILITY 2 — CHRONIC MEDICATION MANAGEMENT

- Search

- Subscribe
- Upload Prescription
- Admin Approval
- Active Subscription
- Automatic Refill
- Delivery
- Repeat

- The application must also intelligently handle unavailable medicines:

- Medicine Out of Stock

- ↓

- Find Same Composition

- +

- Same Strength

- ↓

- Show Alternative Brands

- ↓

Patient Chooses

↓

Add to Cart OR Subscribe

□ The key differentiating features of Auto-Meds are:

- Complete medicine catalog
- Medicine search
- Real-time stock status
- Same composition + same strength alternative brands
- One-time shopping
- Add to Cart
- Checkout
- Delivery management
- Prescription upload
- Prescription approval
- Chronic medication subscriptions
- Automatic refill calculation
- 5-day refill buffer
- Prescription expiry tracking
- Stock-out detection
- Alternative medicine detection during refill
- Subscription cancellation
- Subscription renewal
- Optional pause/resume
- Admin inventory management
- Admin subscription management
- Admin prescription review
- Admin order management
- Patient notifications
- Role-based authentication
- JWT security

The architecture must clearly separate:

□ CART

↓

ONE-TIME ORDER

□ from:

□ SUBSCRIPTION

↓

AUTOMATIC REFILL ORDER

□ while allowing both systems to use the same:

□ Medicine

Inventory

Patient

Order

□ infrastructure.

The final application should feel like a real production-style E-Pharma platform rather than a simple CRUD project.

Start by giving me the **complete architecture, ER diagram, database schema, entity relationships, API design, Angular structure, and all major workflows.**

After that, implement the project step-by-step, starting with the Spring Boot backend and MySQL integration, followed by the Angular frontend.